

Lab3 实验报告

一、思考题

1、对物理地址和虚拟地址的理解

- 请结合MOS中的页目录自映射应用解释代码中 `e->env_pgdir[PDX(UVPT)] = PADDR(e->env_pgdir) | PTE_V` 的含义。

`PADDR(e->env_pgdir)` 是把当前进程页目录的内核虚拟地址转化为物理地址，并且将有效位置一。而 `PDX(UVPT)` 是获取 UVPT (User Virtual Page Table) 的页目录索引，这样就将用户环境的页目录地址写入了页目录中索引为 `PDX(UVPT)` 的位置，从而实现了页目录自映射，使得用户空间可以通过 UVPT 访问自己的页表结构。

2、data 的作用

- `elf_load_seg` 以函数指针的形式，接受外部自定义的回调函数 `map_page`。请你找到与之相关的 `data` 这一参数在此处的来源，并思考它的作用。没有这个参数可不可以？为什么？

```
//kern/env.c
static void load_icode(struct Env *e, const void *binary, size_t size) {
    // ...
    ELF_FOREACH_PHDR_OFF (ph_off, ehdr) {
        Elf32_Phdr *ph = (Elf32_Phdr *) (binary + ph_off);
        if (ph->p_type == PT_LOAD) {
            panic_on(elf_load_seg(ph, binary + ph->p_offset, load_icode_mapper,
e));
        }
    }
    // ...
}
```

观察源码发现显然不能缺少 `data` 参数，因为回调函数 `load_icode_mapper` 函数需要使用 `data` 参数来获得当前的进程环境来加载 `elf` 文件到正确的虚拟环境上。这样的实现保证了函数通用性，使 `elf_load_seg` 不依赖于具体环境实现，并且支持多种映射环境。

3、elf_load_seg 的作用

- 结合 `elf_load_seg` 的参数和实现，考虑该函数需要处理哪些页面加载的情况。

1、非页面对齐的初始部分加载，当段的虚拟地址 (`p_vaddr`) 不是页面对齐时，需要特殊处理第一个不完整的页

```
u_long offset = va - ROUNDDOWN(va, PAGE_SIZE);
if (offset != 0) {
    map_page(data, va, offset, perm, bin, MIN(bin_size, PAGE_SIZE - offset))
}
```

2、完整页面的文件内容加载，按页面对齐方式处理。

```
for (i = ...; i < bin_size; i += PAGE_SIZE) {
    map_page(data, va + i, 0, perm, bin + i, MIN(bin_size - i, PAGE_SIZE))
}
```

3、内存大小超过文件大小的零填充

```
while (i < sgsize) {
    map_page(data, va + i, 0, perm, NULL, MIN(sgsize - i, PAGE_SIZE))
}
```

4、异常处理

```
if ((r = map_page(...)) != 0) {
    return r;
}
```

4、EPC 的含义

- 你认为这里的env_tf.cp0_epc存储的是物理地址还是虚拟地址？

显然是虚拟地址，在使用 MMU 的系统中，CPU 看到的 PC 值始终是虚拟地址。

5、异常处理函数的实现位置

- 试找出0、1、2、3号异常处理函数的具体实现位置。8号异常（系统调用）涉及的do_syscall()函数将在Lab4中实现。

```
# kern/genex.S/handle_int
NESTED(handle_int, TF_SIZE, zero)
    mfc0    t0, CP0_CAUSE
    mfc0    t2, CP0_STATUS
    and     t0, t2
    andi    t1, t0, STATUS_IM7
    bnez    t1, timer_irq
timer_irq:
    li      a0, 0
    j       schedule
END(handle_int)
# kern/genex.S/handle_tlb, handle_mod,
BUILD_HANDLER tlb do_tlb_refill

#if !defined(LAB) || LAB >= 4
BUILD_HANDLER mod do_tlb_mod
BUILD_HANDLER sys do_syscall
#endif

BUILD_HANDLER reserved do_reserved
```

6、时钟的设置

- 阅读entry.S、genex.S和env_asm.S这几个文件，并尝试说出时钟中断在哪些时候开启，在哪些时候关闭。

从内核态返回用户态或用户态执行时期时钟中断被开启；进入内核态时或在内核态执行时期时钟中断被关闭。

7、进程的调度

- 阅读相关代码，思考操作系统是怎么根据时钟中断切换进程的。

一旦时钟中断产生，就会触发4KC硬件的异常中断处理流程。系统将PC指向0x80000180，跳转到.text.exc_gen_entry代码段执行。对于时钟引起的中断，通过.text.exc_gen_entry代码段的分发，最终会调用handle_int函数进行处理。handle_int函数根据Cause寄存器的值判断是否是Timer对应的7号中断位引发的时钟中断，如果是，则执行中断服务函数timer_irq，跳转到schedule中执行。schedule函数来实现具体的进程切换。

二、难点分析

我觉得本次实验第一个难点是恰当的使用panic_on和try两个函数，虽然它们都是用于异常处理，但对于一些异常需要进行进一步处理，有的应该直接输出错误及时反馈。并且try会直接返回，而panic_on执行完后会继续执行原函数。

第二个难点应该就是补充env_alloc函数，像这种有关分配的函数向来都比较复杂，要考虑的细节很多，如果不是有指导书的提示，很难将所有情况考虑周全。首先要判断进程空闲链表是否为空，不为空的话取出第一个进程进行地址空间初始化，之后对进程的变量赋值，最后将这个进程从空闲链表中移除。

第三个难点就是完成schedule函数，首先不能漏掉进程切换的条件，然后还要考虑当前进程仍处于就绪状态时需要移动到调度队列的尾部。之后取出并切换到新的进程。处理完之后，对当前的进程时间片减一，开始执行该进程。

三、实验体会

这次实验的主要内容是操作系统的进程与异常管理。整体的难度个人感觉比第二次实验简单一些。因为不像上次对内存管理和分布一无所知，也没有上次实验复杂的链表宏定义。整体跟着指导书一步步完成，还是能比较清晰地梳理出整个进程管理和异常处理的过程，从进程的初始化、创建、切换到异常、中断的处理，都有了一定的认识和理解。但是，这个单元的实验题数目又变多了，从Lab1的7道题到Lab2的10道题，再到这次Lab3的12道题，题量日益增加，希望下次实验题目数量不要变得更多了。

四、原创说明

- 本文部分内容在LLM（大语言模型）的协助下完成